

02-614 Week 1

String Algorithms

Aidan Jan

August 25, 2026

Strings

A string is a data structure that represents a sequence of letters drawn from an alphabet. Some examples:

- “abcdefg”
- “hello”
- “ACCGTACGG...”

This course covers algorithms used for processing strings.

Exact Matching

One of the first questions we might ask is where a string exists in another string, if it exists. Formally, given a string T and a string P , report all places in T where P occurs.

Naïve Algorithm

```
for every pos i in T:
    for every pos j in P:
        check if T[i + j] = P[j]
```

This algorithm essentially checks every position in the string T to see if P begins at that position. This has a running time of $O(|T| \times |P|)$, which is slow. Ideally, we want an algorithm with $O(|T| + |P|) = O(|T|)$ instead, since that is theoretically the fastest time possible.

The Z Algorithm

This algorithm does exact matching in linear time.

Define Z -values. $Z_i(P) :=$ the length of the longest substring starting at i in P that matches a prefix of P . (e.g., a repeated substring in P).

```
def Zmatch(T, P):
    S = concat(P, '$', T)
    print all i for which Z_i = |P|
```

Essentially, if we knew all the Z_i 's, then finding an exact matching is trivial. The only issue now is, how do we compute the Z_i 's?

Computing Z_i Values

Assume we have computed $Z_2, \dots, Z_{k-1}$, and ℓ and r , where $\ell, r :=$ the left and right edges of the rightmost non-zero z_i values computed so far. The section of z -values between ℓ and r are called the z -box. From here we have a few cases:

1. Case $k > r$: The Z -box ends before index k . If this case occurs, we have to brute-force compute Z_k
2. Case $k \leq r$: The Z -box ends after index k . We know that $\ell < k$ since the start of a z -box must be within the part we already computed. Therefore ℓ is at most $k - 1$. We know for a fact that the characters in a Z -box matches the characters at the beginning of the string. Therefore, since k is in our Z -box, there must exist a k' in the same box at the beginning of the string which we have already calculated. Now, define β as the section of the string between k and r . Now we have two subcases:
 - (a) Subcase $Z_{k'} < \beta$: We know that $Z_k = Z_{k'}$.
 - (b) Subcase $Z_{k'} \geq \beta$: If this is true, then $Z_{k'}$ extends past the end of the Z -box. Therefore, we skip to r , and compute from that point using brute force.